The 4-Question Exam Blueprint

- **Predictable Annual Pattern (2021--2025):**
  - Question 1 Foundational Concepts, Sensing & Hardware (Lec 1 & 2)
  - Question 2 Agency, Behaviors, and Schema Theory (Lec 3 & 6)
  - Question 3 Perception, Biological Mechanisms & Behavior Acquisition (Lec 4, 5, 6)
  - Question 4 Potential Fields & Multi-Robot Task Allocation (Lec 7 & MRS)

1.1 Defining Robot and Robotics

- **Robot:** An electromechanical device that is **reprogrammable**, **multifunctional**, and **sensible for the environment**.
- **Robotics:** The study and application of robot technology.
- **Telerobotics:** Robot systems that are operated remotely.

1.2 Automation vs. Robots

- **Industrial Automation:** Machinery designed for a **specific, single task** (e.g. bottling machine, dishwasher). Always better & more efficient for that single task due to optimal design.
- **Robots:** Machinery designed to carry out a **variety of tasks** (e.g. pick-and-place arms, mobile robots, CNC).

1.3 Asimov's Laws of Robotics & Ethical Scenarios

- **Law 1:** May not injure a human being or, through inaction, allow a human to come to harm.
- **Law 2:** Must obey orders from humans, except where conflicting with Law 1.
- **Law 3:** Must protect own existence, unless conflicting with Laws 1 or 2.
- **Law Zero:** May not injure humanity, or through inaction allow humanity to come to harm (**overrides Laws 1, 2, 3**).

Exam Scenarios:
- **Kill President: Refuse order.** Law 1 strictly prohibits injuring a human; Law 2 obeys orders except when conflicting with Law 1.
- **Kill Terrorist with Bomb in Crowd: Obey & neutralize terrorist.** Inaction harms crowd (violates Law 1). Conflict resolved by **Law Zero** to protect humanity.

1.4 Proprioceptive vs. Exteroceptive Sensing

- **Proprioceptive (Internal):** Monitors robot's own internal mechanical/electrical states (battery level, joint angle, velocity, orientation).
  - **Examples:** Wheel encoders (odometry), gyroscopes, accelerometers, tachometers.
- **Exteroceptive (External):** Measures parameters of surrounding environment (obstacles, light, temp, sound).
  - **Examples:** Cameras, LIDAR, sonar/ultrasound, contact whiskers.
- **Case Study (Contaminated Zone):**
  - **Internal:** Battery voltage, wheel encoders (displacement).
  - **External:** LIDAR/sonar (obstacles), GPS (coordinates).

1.5 Active vs. Passive Sensing

- **Active Sensors:** Emit physical energy into environment & measure reflection/delay.
  - **Examples:** Sonar (sound waves), LIDAR (laser light), IR proximity (IR pulses).
- **Passive Sensors:** Passively capture naturally occurring ambient energy.
  - **Examples:** CCD/CMOS cameras (ambient light), microphones (ambient sound), bump switches (mechanical deflection).

1.6 Deep Dive: Specialized Robotic Sensors

- **A. Touch / Feelers:** Binary limit switches; robust contact detection.
- **B. Inductive vs Capacitive:**
  - **Inductive:** Senses **metallic/conductive objects only** via electromagnetic field oscillation distortion.
  - **Capacitive:** Senses **all materials** (metals, plastics, liquids, glass) via dielectric permittivity shift.
- **C. Infrared (IR):**
  - **Reflectance:** Emitter + detector; binary line tracking.
  - **Triangulation:** PSD sensor; calculates distance via geometric angle. Vulnerable to surface color/ambient light.
- **D. LIDAR:** Rotating laser pulsed ToF ($d = c\frac{t}{2}$). Millimeter 2D/3D angular accuracy.
- **E. Ultrasonic (Sonar):** Time-of-flight sound pulse:

Formula $d = \frac{v \cdot t}{2} \quad (v \approx 340 \text{ m/s})$

  - **Weaknesses:** Wide bearing uncertainty ($\approx 30^\circ$), specular reflection (false negatives), sound latency.

1.7 Robot Programming Methodologies

- **A. Online Programming (Direct):**
  - **Teach Pendant / Lead-Through:** Operator manually guides robot to waypoints; robot records coordinates.
  - **Pros:** Simple, intuitive, no CAD model needed.
  - **Cons:** Halts production line during teaching (downtime).
- **B. Offline Programming (Software):**
  - Programmed in 3D simulator/IDE and uploaded to robot.
  - **Pros:** Zero production downtime, complex logic support.
  - **Cons:** Real-world calibration errors (sim-to-real gap).

1.8 EV3 Classroom Block Diagram Logic

Step 1: Avoid Collision (Binary Bumper):
- Touch Sensor pressed $\rightarrow$ Reverse motor + Turn 90°.

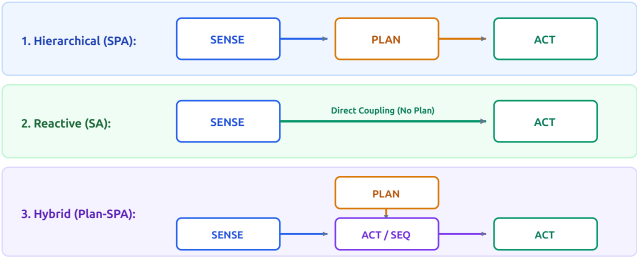
Step 2: Avoid Collision (Ultrasonic Range):
- Ultrasonic distance < 20 cm $\rightarrow$ Steer away.

Step 3a: Line Tracking (Two State Relay):
- Color Sensor on Black ($< 50\%$) $\rightarrow$ Steer Left; White ($\geq 50\%$) $\rightarrow$ Steer Right (Zig-zag line edge follow).

💡 HIGH-YIELD EXAM DISTINCTION (QUESTION 1)

Inductive vs Capacitive: Inductive = **Metals only**; Capacitive = **All materials** (dielectric change). **Ultrasonic vs LIDAR:** Ultrasonic has **wide beam spread** (30°) and **specular reflections**; LIDAR gives precise millimeter angular sweeps. **Asimov Law Zero:** Overrides Laws 1, 2, and 3 to protect **humanity as a whole**.

2.1 Robotic Architectural Paradigms



- **1. Hierarchical / Deliberative (SPA):**
  - Sense → Plan → Act. Builds monolithic **Global World Model**.
  - **Limitation (Frame Problem):** Slow model updates; fragile to environment changes (e.g. **Shakey** with STRIPS).
- **2. Reactive (SA):**
  - Sense → Act directly with **no planning layer**. Vertical decomposition with parallel behaviors.
  - **Subsumption:** Higher layers suppress/inhibit lower layers (Brooks).
- **3. Hybrid (Plan-SPA):**
  - 3-Layer: Deliberative → Sequencer → Reactive execution.

2.2 Marr's Three Information-Processing Levels

- **Level 1: Computational (Theory): What** is the mathematical goal of the task?
- **Level 2: Algorithmic (Representation): How** is it represented (inputs, outputs, decision rules)?
- **Level 3: Implementational (Physical): How** is it physically constructed? (neurons vs microcontrollers/motors).

Case Study: Mosquito vs Search & Rescue Robot:
- **Mosquito:** L1 = Find warm host; L2 = Thermotaxis + Chemotaxis; L3 = Thermal pit organs, biological brain, flight muscles.
- **S&R Robot:** L1 = Find survivors in rubble; L2 = Vector sum of FLIR thermal + CO_2 gas centroids; L3 = FLIR sensor, ARM MCU, DC motors.
- Note: Levels 1 & 2 are abstract; **only Level 3 differentiates biology from robots**.

2.3 Biological Inspiration to Robotic Mappings

Biological System	Robotic Mapping
Bat Echolocation	Active ultrasonic distance sensors
Ant Foraging	Ant Colony Optimization (ACO) shared maps
Bird Flocking	Boids (Cohesion, Alignment, Separation) UAVs
Frog's Tongue Reflex	Visual servoing high-speed robotic arm
Cuttlefish Camouflage	Pattern mapping on flexible E-skins

2.4 Biological Behavior Classifications

- **1. Reflexive Behavior:** Pure stimulus-response ($S \to R$) with no memory.
  - **Taxes:** Orienting relative to stimulus (e.g. phototaxis to light).
  - **Fixed-Action Pattern (FAP):** Response continues much longer than stimulus (e.g. web spinning, courtship dance).
- **2. Reactive Behavior:** Learned through repetition; automatic "muscle memory" (e.g. bike riding).
- **3. Conscious Behavior:** Deliberative, highly cognitive (e.g. path planning, assembly).

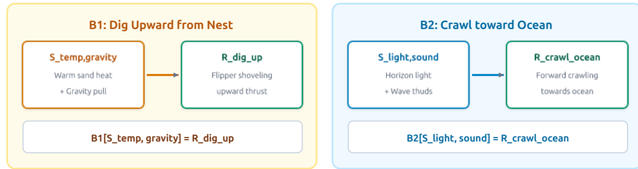
2.5 Schema Theory in OOP Robotics

- **Behavioral Schema:** Modeled in OOP as a class with exactly one **Perceptual Schema** and one **Motor Schema**.
  - **Perceptual Schema:** Extracts percepts from raw sensors.
  - **Motor Schema:** Generates action vector from percept.
  - **Schema Instantiation (SI):** Parameterizing generic class template with runtime properties (speed, color).

Mathematical S-R Schema Notation:

Notation

$$\{B : S \to R\} \quad \text{or} \quad B[S] = R$$



Case Study: Baby Turtle Nest Hatching:
- **B1 (Dig Upward):**
  - S: Warm sand temp gradient + gravity pull (negative geotaxis).
  - R: Upward flipper shoveling.
  - $B_1[S_{\text{temp, gravity}}] = R_{\text{dig\_up}}$.
- **B2 (Crawl to Ocean):**
  - S: Bright horizon light + wave acoustic thuds.
  - R: Crawling thrust towards ocean.
  - $B_2[S_{\text{light, acoustics}}] = R_{\text{crawl\_to\_ocean}}$.

💡 **EXAM FORMULA NOTATION (QUESTION 2)**

Schema Notation: Always write both $B[S] = R$ and $\{B : S \to R\}$. **Marr's 3 Levels:** Level 1 = Goal/Theory; Level 2 = Rules/Transforms; Level 3 = Hardware implementation.

Q3 Question 3: Perception, Biological Mechanisms & Behavior Acquisition

PERCEPTION & IRM

3.1 Action-Perception Cycle & Affordances

- **Action-Perception Cycle:** Cyclic feedback loop: Action modifies environment $\to$ Sensed by agent $\to$ Determines next Action.
- **Gibson's Ecological Approach:** Perception is direct, active, and environmental (not passive pixel reconstruction).
- **Affordances:** Actionable properties directly perceivable between environment and agent (e.g. a chair affords "sitting", a door handle affords "pulling").

3.2 Four Methods of Acquiring Behaviors

Acquisition Method	Operating Principle
1. Ethologically Guided	Hard-coding innate animal behaviors into robot schemas.
2. Supervised Learning	Training on expert human operator labeled datasets.
3. Reinforcement Learning	Trial-and-error optimization via reward/penalty (Q-learning).
4. Imitation Learning	Direct sensorimotor mapping by observing demonstrations.

3.3 & 3.4 Innate Releasing Mechanisms (IRM)

- **IRM Definition:** Neural biological circuit that acts as a gate/switch: activates a **Fixed-Action Pattern (FAP)** upon detecting a specific **Sign Stimulus / Releaser**.
- **Architectures:**
  - **Combinatorial (Boolean):** Logical gating (A AND B).
  - **Sequential (Chain):** Precondition of behavior N satisfied by completion of behavior $N-1$.

Scenario A: Arctic Tern Beak Pecking

- **Context:** Chick pecks red spot on parent beak to trigger feeding.
- **IRM Latch:** Releaser = Visual Red Spot moving. Action = Feed chick.

```
enum Releaser { PRESENT, NOT_PRESENT };

void ParentBirdControlLoop() {
  while (TRUE) {
    if (DetectChickPeckOnRedSpot() == PRESENT) {
      RegurgitateFood();
      FeedChick();
    } else {
      MaintainNest();
    }
  }
}
```

Scenario B: Honeybee Waggle Dance

- **Context:** Dance angle = food direction relative to sun; waggle duration = distance.
- **Compound Releasers:** Energy OK AND Weather OK AND Dance decoded.

```
void ForagerBeeControlLoop() {
  while (TRUE) {
    if (IsEnergySufficient() == PRESENT) {
      if (IsWeatherSafe() == PRESENT) {
        if (DecodeDanceAngle(&ang, &dist)) {
          FlyToTarget(ang, dist);
          ExtractNectar();
          ReturnToHive();
        } else { SearchRandomly(); }
      } else { StayInHive(); }
    } else { RestInHive(); }
  }
}
```

Scenario C: Ant Colony Trail Following

- **Context:** Scout leaves pheromones; followers follow if energy OK, no alarm danger, and trail strength $\geq \text{THRESHOLD}$.

```
void FollowerAntControlLoop() {
  while (TRUE) {
    if (IsEnergySufficient() == PRESENT) {
      if (IsDangerSignal() == NOT_PRESENT) {
        if (Pheromone() == PRESENT &&
            GetStrength() >= THRESHOLD) {
          FollowTrailToFood();
          BringFood();
          ReinforceTrail();
        } else { SearchRandomly(); }
      } else { StayInNest(); }
    } else { StayInNest(); }
  }
}
```

💡 EXAM PSEUDOCODE RULES (QUESTION 3)

Nested IF Gates: Always structure IRM pseudocode using explicit nested `if (Releaser == PRESENT)` blocks to reflect biological threshold gating. **Compound Releasers:** Check Energy o Environmental Danger o Signal Stimulus.

Question 4: Behavioral Coordination & Multi-Robot Systems

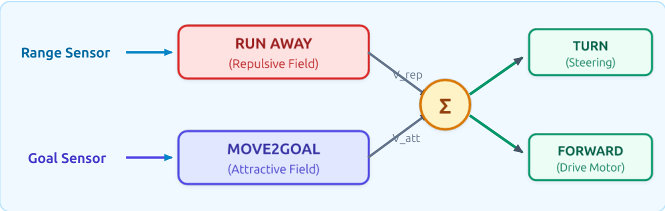
4.1 Cooperating vs. Competing Behaviors

- **Concurrent Competing (Arbitration):** Winner-take-all behavior selection. Lower-priority behaviors are suppressed by higher layers (e.g. **Subsumption**).
- **Concurrent Cooperating (Blending):** Outputs of multiple behaviors are mathematically combined into a smooth action vector (e.g. **Motor Schema Potential Fields**).

4.2 Potential Fields (PFields) Methodology

- **Concept:** Robot modeled as a charged particle. Obstacles are **repulsive sources**; goals are **attractive sinks**.
- **Vector Summation ($Sigma$):** Each motor schema outputs a vector (m, d); final force is Σ:

Resultant Force $\boldsymbol{V}_{\text{resultant}} = \boldsymbol{V}_{\text{attractive}} + \boldsymbol{V}_{\text{repulsive}}$



4.3 PField Magnitude Profiles & Math Formulations

- **1. Constant Profile:** Constant velocity inside D, zero outside. **Drawback:** Jerky boundary oscillations.
- **2. Linear Drop-Off Profile ($y = m x + b$):**
  - Repulsive Field (Away from Obstacle):

$$V_{\text{dir}} = -180^{\circ}, \quad V_{\text{mag}} = \begin{cases} \frac{D-d}{D} & \text{for } d \le D \\ 0 & \text{for } d > D \end{cases}$$

  - Attractive Field (Towards Goal):

$$V_{\text{dir}} = \text{angle to goal}, \quad V_{\text{mag}} = \begin{cases} 1 & \text{for } d > D \\ \frac{d}{D} & \text{for } d \le D \end{cases}$$

- **3. Exponential Drop-Off Profile:**

Exponential $V_{\text{mag}} = e^{-k \cdot d^2}$

4.4 The Local Minima Problem & Solutions

- **Deadlock Cause:** Equal magnitude, opposite direction vectors sum to zero ($\boldsymbol{V}_{\text{res}} = \boldsymbol{0}$).
- **Three Key Solutions:**
  - **1. Random Noise / Perturbation:** Heuristic random push; simple but sub-optimal.
  - **2. Harmonic Functions ($\nabla^2 \varphi = 0$):** Mathematically guarantees no local minima; computationally heavy.
  - **3. Navigation Templates (NaTs) / Informed Redirection:** Feeds strategic goal direction to route around obstacle.

4.5 Multi-Robot Systems (MRS) Foraging Strategies

Foraging Strategy	Principle & Trade-offs
Random Locomotion	Wander randomly. Simple, zero comms. Slowest, collision crashes on scale.
Explicit (Fixed)	Equal spatial division. Complete coverage, no crosstalk. Inefficient if clustered.
Implicit (Repel)	Omnidirectional repel beacons. Great for clusters, no GPS needed.
Beacon Recruitment	Stationary beacon upon food discovery. Fastest for clustered items.

💡 HIGH-YIELD EXAM FORMULAS (QUESTION 4)

Repulsive Linear Profile: $V_{\text{mag}} = \frac{D-d}{D}$ with direction -180°. **Attractive Linear Profile:** $V_{\text{mag}} = \frac{d}{D}$ (slows down to prevent overshoot). **Local Minima Trap:** Occurs when $\boldsymbol{V}_{\text{att}} + \boldsymbol{V}_{\text{rep}} = \boldsymbol{0}$.

Scenario 1: Robotics 2022 (Ukrainian Core Recovery)

- **Given Data:** Rewards: $X = 120, Y = 180, Z = 200$. Transitions: $X \leftrightarrow Y = 60, Y \leftrightarrow Z = 50$. Initial Dist: A to $X = 50, Y = 100, Z = 110$; B to $X = 70, Y = 60, Z = 75$; K to $X = 200, Y = 200, Z = 200$. K sells $Y + Z$ data for 20.

Robot	X	Y	Z	X+Y	Y+X	Y+Z	Z+Y	X+Y+Z	Z+Y+X
A	50	100	110	110	160	150	160	160	220
B	70	60	75	130	120	110	125	180	185
K	200	200	200	260	260	250	250	310	310

- **Step (ii) Allocation & System Cost:**
  - **Opt 1:** A wins X (Cost 50), B wins $Y + Z$ (Cost 110, pays 20 to K). $\mathrm{SC} = 50 + 110 = \mathbf{160}$.
  - **Opt 2:** A wins $X + Y$ (Cost 110), B wins Z (Cost 75). $\mathrm{SC} = 110 + 75 = 185$.
  - **Allocation: Robot A → X, Robot B → $Y + Z$, Final System Cost = 160.**
- **Step (iii) Net Profit Calculation:**
  - **Robot A:** $\mathrm{Profit} = 120 - 50 = \mathbf{70}$.
  - **Robot B:** $\mathrm{Profit} = 180 + 200 - 110 - 20 = \mathbf{250}$.
  - **Robot K:** $\mathrm{Profit} = \mathbf{20}$ (sells data).
  - **Verification:** $\text{System Profit} = 500 - 160 = \mathbf{340} = 70 + 250 + 20$.

Scenario 2: Robotics 2023 (Landslide Deployment)

- **Given Data:** Rewards: $X = 120, Y = 180, Z = 200$. Transitions: $X \leftrightarrow Y = 60, Y \leftrightarrow Z = 50$. Initial Dist: A to $X = 50, Y = 100, Z = 110$; B to $X = 110, Y = 75, Z = 60$; K to $X = 200, Y = 200, Z = 200$. K sells $Y + Z$ for 20.

Robot	X	Y	Z	X+Y	Y+X	Y+Z	Z+Y	X+Y+Z	Z+Y+X
A	50	100	110	110	160	150	160	160	220
B	110	75	60	170	135	125	110	185	170
K	200	200	200	260	260	250	250	310	310

- **Step (ii) Allocation & System Cost:**
  - **Opt 1:** A wins X (Cost 50), B wins $Z + Y$ (Cost 110, pays 20 to K). $\mathrm{SC} = 50 + 110 = \mathbf{160}$.
  - **Opt 2:** A wins $X + Y$ (Cost 110), B wins Z (Cost 60). $\mathrm{SC} = 110 + 60 = 170$.
  - **Allocation: Robot A → X, Robot B → $Z + Y$, Final System Cost = 160.**
- **Step (iii) Net Profit Calculation:**
  - **Robot A:** $\mathrm{Profit} = 120 - 50 = \mathbf{70}$.
  - **Robot B:** $\mathrm{Profit} = 180 + 200 - 110 - 20 = \mathbf{250}$.
  - **Robot K:** $\mathrm{Profit} = \mathbf{20}$. Verification: $\text{Team Profit} = 500 - 160 = \mathbf{340}$.

Scenario 3: Robotics 2024 (Storm-hit Backup Mission)

- **Given Data:** Rewards: $X = 150, Y = 200, Z = 300$. Transitions: $X \leftrightarrow Y = 30, Y \leftrightarrow Z = 40$. Dist: A: $50, 100, 150$; B: $150, 50, 20$; K: $350, 320, 310$. K sells $X + Y$ for 60, $Y + Z$ for 50.

Robot	X	Y	Z	X+Y	Y+X	Y+Z	Z+Y	X+Y+Z	Z+Y+X
A	50	100	150	80	130	140	190	120	220
B	150	50	20	180	80	90	60	220	90
K	350	320	310	380	350	360	350	420	380

- **Step (ii) Allocation & System Cost:**
  - **Opt 1:** A wins X (50), B wins $Y + Z$ ($50 + 40 = 90$, pays 50). $\mathrm{SC} = 50 + 90 = 140$.
  - **Opt 2:** A wins $X + Y$ ($50 + 30 = 80$, pays 60), B wins Z (20). $\mathrm{SC} = 80 + 20 = \mathbf{100}$.
  - **Allocation: Robot A → $X + Y$ (Cost 80, pays 60), Robot B → Z (Cost 20), System Cost = 100.**
- **Step (iii) Net Profit Calculation:**
  - **Robot A:** $\mathrm{Profit} = 150 + 200 - 80 - 60 = \mathbf{210}$.
  - **Robot B:** $\mathrm{Profit} = 300 - 20 = \mathbf{280}$.
  - **Robot K:** $\mathrm{Profit} = \mathbf{60}$. Verification: $\text{Team Profit} = 650 - 100 = \mathbf{550}$.

Scenario 4: Robotics 2025 (Interstellar Probe 3i/Atlas)

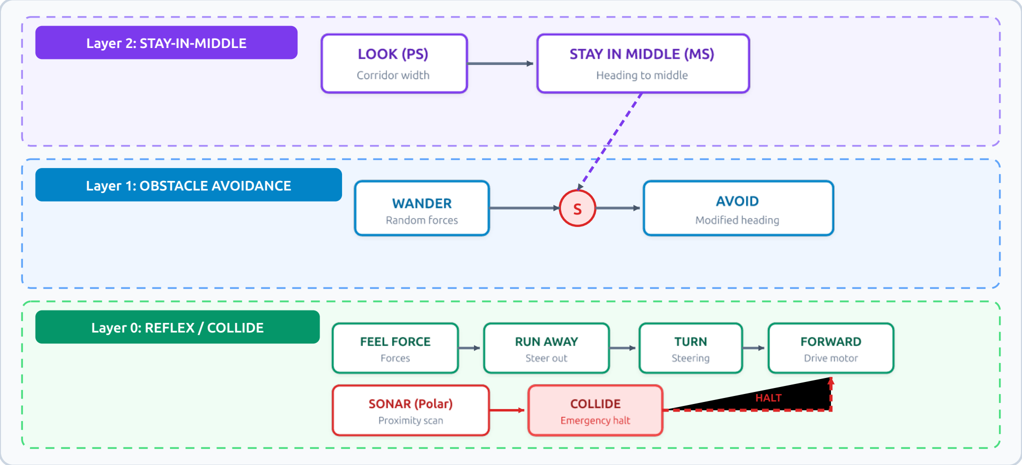
- **Given Data:** Rewards: $X = 150, Y = 150, Z = 150$. Transitions: $X \leftrightarrow Y = 50, Y \leftrightarrow Z = 40$. Dist: A: $50, 100, 120$; B: $80, 40, 100$; K: $300, 200, 140$. K sells 0 data.

Robot	X	Y	Z	X+Y	Y+X	Y+Z	Z+Y	X+Y+Z	Z+Y+X
A	50	100	120	100	150	140	160	140	210
B	80	40	100	130	90	80	140	170	190
K	300	200	140	350	250	240	180	390	230

- **Step (ii) Allocation & System Cost:**
  - **Opt 1:** A wins X (Cost 50), B wins $Y + Z$ (Cost $40 + 40 = 80$). $\mathrm{SC} = 50 + 80 = \mathbf{130}$.
  - **Opt 2:** A wins $X + Y$ (Cost 100), B wins Z (Cost 100). $\mathrm{SC} = 100 + 100 = 200$.
  - **Allocation: Robot A → X, Robot B → $Y + Z$, Final System Cost = 130.**
- **Step (iii) Net Profit Calculation:**
  - **Robot A:** $\mathrm{Profit} = 150 - 50 = \mathbf{100}$.
  - **Robot B:** $\mathrm{Profit} = 150 + 150 - 80 = \mathbf{220}$.
  - **Robot K:** $\mathrm{Profit} = \mathbf{0}$. Verification: $\text{Team Profit} = 450 - 130 = \mathbf{320}$.

D1: Robot Subsumption Navigation Flowchart (Stay-In-Middle)

- **Architecture:** Rodney Brooks' Subsumption Architecture for corridor navigation with 3 horizontal layers:
  - **Layer 2 (STAY-IN-MIDDLE):** `LOOK (PS)` feeds corridor width to `STAY IN MIDDLE (MS)` which outputs heading to middle.
  - **Layer 1 (OBSTACLE AVOIDANCE):** `WANDER` outputs random forces; `AVOID` suppresses `WANDER` via circle `S` (Suppression).
  - **Layer 0 (REFLEX/COLLIDE):** `FEEL FORCE` → `RUN AWAY`, `TURN`, `FORWARD`. `SONAR` polar plot triggers high-priority `HALT` to `FORWARD`.



D2: Potential Field Local Minima Trapping Grid

- **2D Vector Navigation Grid:**
  - Robot node R on left; Goal node G on right; Obstacle node O in direct straight-line path.
  - Vectors far from obstacle point directly towards G.
  - Vectors near obstacle diverge around O.
  - Directly in front of obstacle, opposing vectors meet:

$$\text{Local Minima Condition} \quad V_{\text{attractive}} + V_{\text{repulsive}} = 0$$

  - The net force is zero; the robot is trapped in deadlock.